

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, transforms
from PIL import Image
import matplotlib.pyplot as plt
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
loader = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor()
])

def load_image(path):
    image = Image.open(path).convert("RGB")
    image = loader(image).unsqueeze(0)
    return image.to(device, torch.float)
```

```
content_img = load_image("/content/content_image.jpg")
style_img = load_image("/content/style_image painting.jpg")
```

```
vgg = models.vgg19(pretrained=True).features.to(device).eval()
```

```
for param in vgg.parameters():
    param.requires_grad = False
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pr
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other t
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoi
100%|██████████| 548M/548M [00:08<00:00, 70.5MB/s]
```

```
def get_features(image, model):
    layers = {
        '0': 'conv_1',
        '5': 'conv_2',
        '10': 'conv_3',
        '19': 'conv_4',
        '28': 'conv_5'
    }

    features = {}
    x = image

    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features
```

```
def gram_matrix(tensor):
    b, c, h, w = tensor.size()
    tensor = tensor.view(c, h * w)
    return torch.mm(tensor, tensor.t())
```

```

content_features = get_features(content_img, vgg)
style_features = get_features(style_img, vgg)

style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features}

```

```
target = content_img.clone().requires_grad_(True).to(device)
```

```

optimizer = optim.Adam([target], lr=0.003)

steps = 300
content_weight = 1e4
style_weight = 1e2

for i in range(steps):

    target_features = get_features(target, vgg)

    # Content Loss
    content_loss = torch.mean(
        (target_features['conv_4'] - content_features['conv_4'])**2
    )

    # Style Loss
    style_loss = 0

    for layer in ['conv_1', 'conv_2', 'conv_3', 'conv_4', 'conv_5']:
        target_feature = target_features[layer]
        target_gram = gram_matrix(target_feature)
        style_gram = style_grams[layer]

        _, c, h, w = target_feature.shape

        style_loss += torch.mean((target_gram - style_gram)**2) / (c * h * w)

    # Total Loss
    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    if i % 50 == 0:
        print(f"Step {i}, Loss: {total_loss.item()}")

```

```

Step 0, Loss: 1761.7193603515625
Step 50, Loss: 973.9031372070312
Step 100, Loss: 895.9078369140625
Step 150, Loss: 857.0850219726562
Step 200, Loss: 830.8939208984375
Step 250, Loss: 811.4954833984375

```

```

def imshow(tensor):
    image = tensor.cpu().clone().detach()
    image = image.squeeze(0)
    image = transforms.ToPILImage()(image)
    plt.imshow(image)
    plt.axis('off')

imshow(target)

```

